Push-T on a UR5e: Zero-Shot Transfer and Self-Improvement

Oct 14, 2026 · Shakthibala Sivagami Balamurugan

Summary

A flow matching policy trained on the 2D Push-T benchmark pushes the T into place with a simulated UR5e arm **without any robot training data**: 0.79–0.82 mean reward on 100 evaluation seeds, the same as in its own 2D simulator (0.817), even though the robot's table has realistic friction and the 2D world has none. Self-improvement by filtered behavior cloning, which retrains on the robot's own successful episodes, did **not** improve on it. Three variants settled at 0.64–0.71 mean reward, below a control trained on the human demonstrations alone (0.74–0.80).

Setup

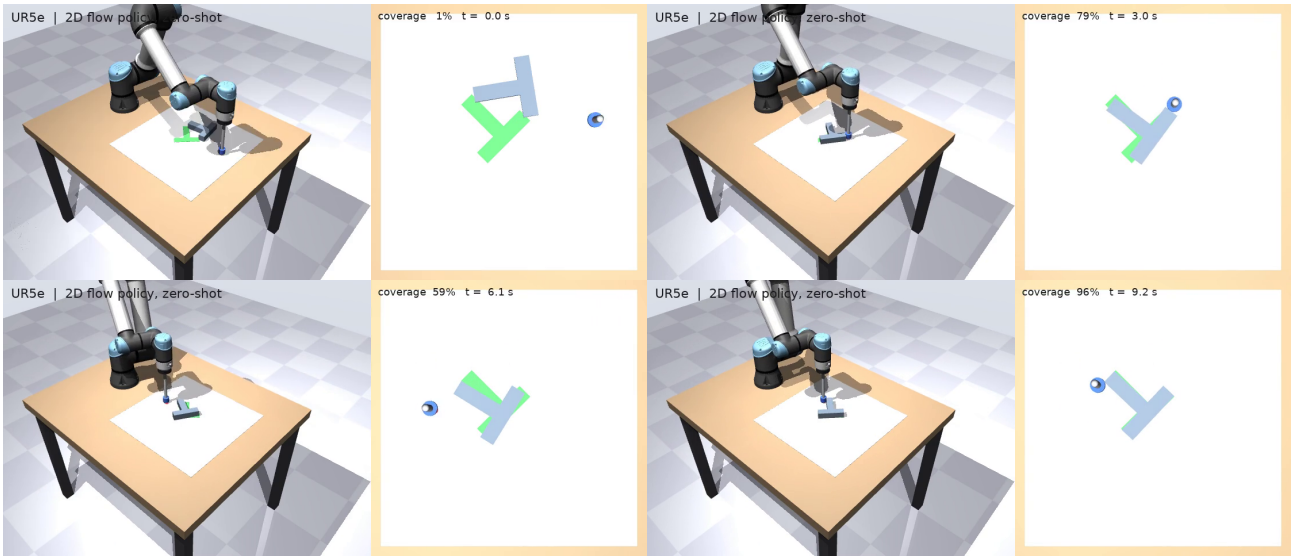


Figure 1: A zero-shot rollout (seed 7, solved in 9.3 s). Left: the UR5e from the front camera. Right: the same moment from above with the arm hidden, Push-T style; the red dot is the policy's current target.

Scene. A UR5e from MuJoCo Menagerie stands on a table and holds a 19 cm stick with a 15 mm-radius tip. The T block (120 mm bar, 90 mm stem, 30 mm thick, 234 g) slides on the table with Coulomb friction $\mu = 0.35$. The 2D benchmark's 512 × 512 px workspace is mapped onto the table at 1 px = 1 mm, so the state (tip x, y and block x, y, angle in pixels), the action (a target tip position in pixels), the random start states per seed, and the success test (more than 95 % of the goal covered) are all identical to the 2D task.

Controller. The policy runs at 10 Hz and outputs 8-step action chunks, executed open-loop. Each target drives a reference point with the 2D agent's PD dynamics (k_p = 100, k_v = 20) at 100 Hz. Damped least-squares inverse kinematics turns the reference into joint targets with the stick held vertical at a fixed tip height, inside the joint limits. Gravity compensation and a velocity feed-forward term cancel the position servos' sag and lag.

Check	Result
Tip tracking on 5 human demo episodes (738 steps)	0.28 px mean, 1.34 px max error
Tip height while pushing	20.0 ± 0.1 mm
Slide after a 0.3 m/s push	13.0 mm (Coulomb prediction 13.1 mm)
Simulation speed	6 ms per 0.1 s control step on one CPU core
T at the goal pose	coverage 1.000

Zero-shot transfer

Both 2D policies from the course assignment ran unchanged on the arm, on the same 100 evaluation seeds as in 2D.

Policy (trained in 2D)	2D Push-T	UR5e, mean reward	UR5e, solved	UR5e, reward < 0.3
MSE	0.629	0.524	7 %	38 %
Flow matching	0.817	**0.79–0.82**	**12–20 %**	10–11 %

The flow policy's range covers four evaluations of the same checkpoint. The rollouts are seeded, but the evaluations ran on different CPU types, and tiny floating-point differences grow during contact. That spread, about ±0.02 mean reward and ±4 points of solved episodes, sets the resolution of every comparison below.

A likely reason the flow policy transfers despite the change in physics is that it re-plans every 8 steps from the observed block pose: when the T stops sliding sooner than it would in 2D, the next chunk simply pushes again. MSE probably loses more because its averaged chunks already stall against the block in 2D, and friction removes the coasting that sometimes rescued them.

Self-improvement

Each round runs the current policy on 2000 fresh seeds (≥ 10,000, never used for evaluation), keeps only the solved episodes, and trains on every success collected so far. Round 0 is the 2D flow policy.

- **A, robot successes, from scratch:** a new policy is trained for 60k steps on robot successes only.
- **B, human + robot, from scratch:** the same, plus the 206 human demonstrations in every round.
- **C, robot successes, fine-tuned:** the previous round's policy is trained 20k more steps at learning rate 1e-4, keeping its normalizer.

A control trains on the human demonstrations alone with the same code for 75.6k steps (2 seeds). It separates the effect of the robot data from the difference between one training run and another.

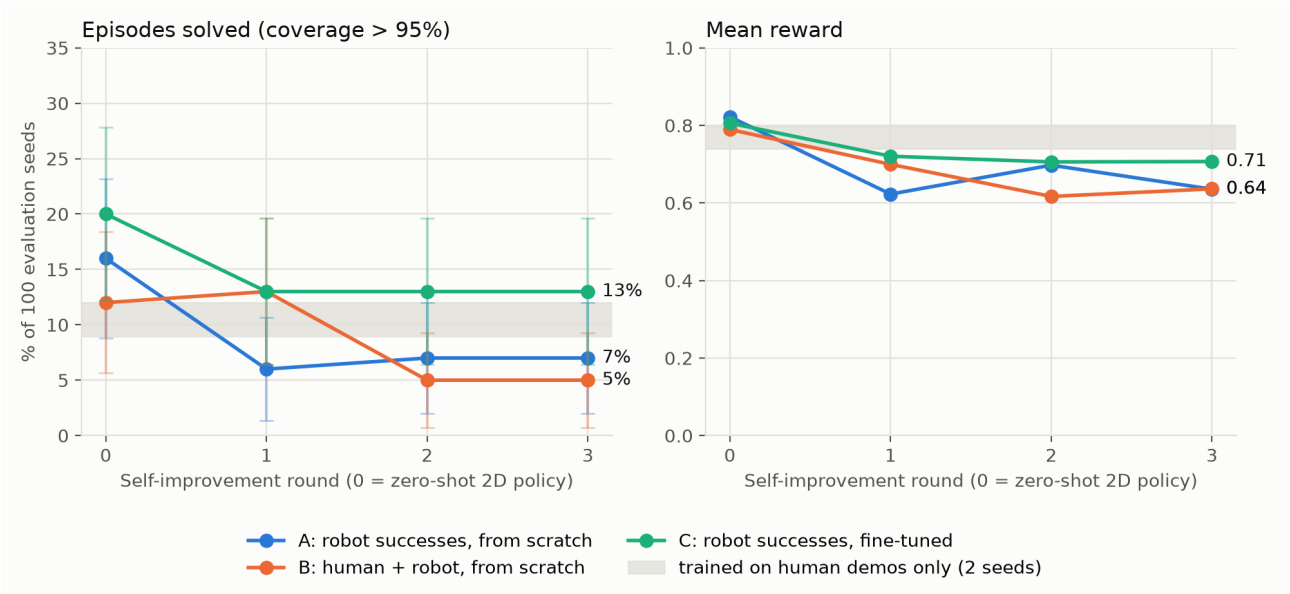


Figure 2: Evaluation after each round. Error bars: 95 % binomial intervals. Gray band: range of the human-only control over 2 seeds.

	Round 1	Round 2	Round 3	Episodes kept per round
A: robot successes, from scratch	0.623 · 6 %	0.697 · 7 %	0.635 · 7 %	290 · 149 · 128
B: human + robot, from scratch	0.699 · 13 %	0.617 · 5 %	0.636 · 5 %	288 · 135 · 93
C: robot successes, fine-tuned	0.720 · 13 %	0.706 · 13 %	0.707 · 13 %	256 · 229 · 235
Control: human demos only	0.798 · 12 % (seed 0)	0.740 · 9 % (seed 1)		

Cells are mean reward · % of the 100 evaluation seeds solved.

No variant beat the control on mean reward. Every round of A and B is 0.04–0.12 below the control's lower end (0.740), beyond the ±0.02 evaluation noise. C is 0.02–0.03 below it, which is about the size of the noise, so fine-tuning roughly held its ground without improving. Solve rates differ by less than their confidence intervals, so the data cannot say whether the robot successes changed how often episodes finish.

Why retraining on robot successes hurt. The kept episodes come from the start states the current policy already solves, so each round's dataset is narrower than the

last, and the from-scratch variants lose behaviour for the hard starts. As those policies weaken, they keep fewer episodes (A: 290 → 128, B: 288 → 93), which narrows the data further. Fine-tuning (C) avoids that spiral: it starts from a policy that still covers the hard starts and keeps about 230–255 episodes per round. It still does not climb, because a solved episode only says "this whole trajectory worked" and never which of its pushes mattered.

What would likely help. Using the reward inside training rather than only as a keep-or-drop filter: reinforcement-learning fine-tuning of the flow policy (for example, policy-gradient fine-tuning of the denoising steps, as in DPPO), or advantage-weighted regression on all collected episodes. Evaluating every comparison on several training seeds and more than 100 episodes would also make differences of a few points measurable.

Engineering notes

- Training crashed on Intel 6767P and AMD EPYC 9654 nodes (glibc heap-corruption assertion) inside a `torch.compile` kernel, but not on the EPYC 7543 node where the kernel cache had been built. The cache lived on shared storage, and a per-host cache directory did not fix it because the cluster's temporary directory is also shared. Robot training therefore runs in eager mode, which is fast enough for a 3 × 256 MLP.
- Headless rendering on the cluster needs the OSMesa software renderer: neither EGL on the CPU nodes nor the GPU nodes' driver offers the device platform MuJoCo requires. Mesa 25 and later no longer ship OSMesa, so the setup pins `mesalib=24.3` and preloads it so its C++ runtime wins over the system's.

Reproduce

Commands are in `ur_pusht/README.md`. Raw per-round metrics are in `ur_pusht/results/`. Each self-improvement variant takes 7–16 minutes on one 32-core CPU node and each control run 3–4 minutes, so all five run in parallel in about a quarter of an hour.